

chrome_tab_organiser — v1.5 Editor

v1.5 Editor — Import Tabs from Live Chrome

chrome_report_edit_v1.5.py

— a local web app to edit generated tab reports

— **new:** copy tabs out of the Chrome you are running, into any report window, keeping groups and colours

chrome_report_v1.4.py

— generate the HTML/JSON report this editor reads

Document suffix `_261005` — self-contained tools, Python standard library only.

Contents

1. What v1.5 adds
2. How the import works
3. What comes across — and what doesn't
4. Where the live tabs come from
5. Files
6. Running the editor
7. Editing, dragging and sorting (unchanged from v1.41)
8. Testing
9. Troubleshooting
10. Relationship to the report generator

New feature: import tabs straight from the running Chrome, into any window of your report **Date suffix:**

`_261005` **Applies to:** `chrome_report_edit_v1.5.py` (editor), `chrome_report_v1.4.py` (report generator)

Self-contained tools — Python standard library only, no dependencies to install.

1. What v1.5 adds

An **Add tabs from window** button sits in every window header of the editor, beside **Add tab**. Click it and you get a live list of what Chrome is actually running right now: its windows and its named tab groups. Pick one and every tab in it is copied into the report window you started from.

Situation	What you used to do	What you do now
Regenerate the report, then open three more tabs	Run the generator again, accept a new file name	Click Add tabs from window , choose the Chrome window
Add one group's tabs to an old report	Type each URL by hand with Add tab	Click Add tabs from window , choose the group
Merge a Chrome window into a report window	Reorder two saved reports by hand	Import, then drag if you still want to rearrange

Chrome can keep running — its session-recovery files are only ever read.

2. How the import works

1. Click **Add tabs from window** in the header of the window that should receive the tabs (the header that already has **Add tab**).
1. The **Add tabs from Chrome** dialog opens with a live list:
 - **Chrome windows** — `Window 1`, `Window 2`, ... each with its tab and group count, and `• active` on the one Chrome is currently showing.
 - **Chrome tab groups** — every named group across all windows, with its tab count, e.g. `ChIP — 24 tabs`.
1. The line under the list previews the result: `""18 tabs will be added,`
`inserted at the beginning of the window.""`
1. Press **Add tabs**.

Why is it called `Window 1` and not `My browser window`? Chrome windows have no user-assigned names — only the report generator asks you for names. So the picker labels them by position, in tab-strip order, and marks the active one. You always choose which report window receives the tabs.

Choosing where the tabs land

"sort groups alphabetically"	Imported tabs go
unticked (default)	to the very beginning of the target window, in Chrome's own tab-strip order
ticked	into place so the window stays sorted : ungrouped first, then each tab alphabetically by its own group

3. What comes across — and what doesn't

Carried over	Reason
Tab title and URL	verbatim copy of what Chrome has
Group name	the tab stays in its group in the report
Group colour	the chip is the same colour as in Chrome

Not carried over:

- The **active-window star**. That mark describes your report's own window, not

Chrome's, so importing never adds one.

Duplicates

A tab whose URL is **already in the target window** is skipped, and the status bar reports both numbers:

```
Added 18 tabs from Window 3 (2 already in this window).
```

The comparison is deliberately loose — letter case, a trailing slash and a `#fragment` are ignored, so these three count as the same page:

```
https://Example.com      https://example.com/      https://example.com#top
```

Two points worth knowing:

- If Chrome really does have the same page open twice, **both** are imported.

Only what is already in the *report* is skipped — the report is what you are avoiding duplicates against.

- Importing the same window a second time therefore adds nothing, because its

URLs are in the window by then.

4. Where the live tabs come from

The editor reads Chrome's session-recovery files — the `SNSS` format written into `<data dir>\Default\Sessions\` — and parses the same records the report generator does. That is the entire mechanism:

- **Chrome does not have to be closed**, and the import does not talk to a

running Chrome. Nothing is closed, added or removed in Chrome.

- **Chrome does not have to be open either**. The files keep the last saved

session, so a closed Chrome works too.

- The newest session file is often **locked while Chrome is writing it**. It is

skipped with a reason and the newest complete snapshot is used instead — close the dialog and click the button again in a moment if the list looks a little stale.

- A malformed or foreign file is skipped rather than reported as an error.

Session files are **read-only** here; no timestamp, byte or handle inside Chrome is modified.

To turn the whole feature off, start the editor with `--no-chrome`. No session file is opened, and the button explains why.

5. Files

File	Role
<code>chrome_report_v1.4.py</code> / <code>.exe</code>	Generates the HTML/JSON report from Chrome's sessions.
<code>chrome_report_edit_v1.5.py</code> / <code>.exe</code>	The editor described here — report editing plus live import.
<code>chrome_report_edit_v1.41.py</code> / <code>.exe</code>	Previous release (drag fix), kept for reference.
<code>test_editor_js.py</code>	Headless browser test of the editor, 58 checks.
<code>build_exe.py</code>	Rebuilds <code>chrome_report_v1.4.exe</code> and <code>chrome_report_edit_v1.5.exe</code> .
<code>tab_organiser_ext\</code>	Optional extension that opens real Chrome windows/groups from the report.

The v1.41 release and its executables are unchanged and still work.

6. Running the editor

```
python chrome_report_edit_v1.5.py [--dir PATH] [--port 8765]
                                [--host 127.0.0.1] [--no-browser]
                                [--no-chrome]
```

Option	Meaning
<code>(none)</code>	Serves the <code>*.html</code> files next to the script/exe and opens the browser.
<code>--dir PATH</code>	Directory the editor manages and saves into.
<code>--port N</code>	Local port (default <code>8765</code>).
<code>--host ADDR</code>	Bind address (default <code>127.0.0.1</code> , loopback only).
<code>--no-browser</code>	Print the URL instead of opening the browser.
<code>--no-chrome</code>	Disable the live import ; no session file is read.

Or with the executable:

```
chrome_report_edit_v1.5.exe --dir reports
```

Everything the editor talks to is `localhost`; nothing goes out to the network.

7. Editing, dragging and sorting (unchanged from v1.41)

- Click a “Window N” heading to rename a window; click a chip, title or URL to

edit it in place; **x** on a row deletes it.

- **+** on a row inserts after it; **Add tab** in a header appends at the end.
- Drag the **↑ grip** in a row's Group cell — up and down inside a window, or

into another window. Drags start only from the grip, so clicking text still edits it. You may release over a chip, another row's grips, a link or a bare cell; releasing over a link places the tab just before that row.

- **Sort groups alphabetically** orders each window: ungrouped first, then

groups A–Z. An actual drag switches it off so your manual order wins; unticking restores the order exactly.

- Rows are renumbered (`data-pos` = 1..N) after every insert, delete, drag,

sort, import and save. Reports that load without `data-pos` keep it that way.

- Nothing touches disk until you press **Save**.

8. Testing

Suite	Scope
<code>python test_editor_js.py</code>	58 headless browser checks: the 33 v1.41 drag/edit/save checks plus 25 import checks — button placement, the live picker, source order, duplicate skipping (including case and trailing slash), group name/colour carry-over, sorted placement, renumbering, and the error path.
native CDP drag	Drives real mouse gestures over Chrome DevTools Protocol so HTML5 drag goes through Chrome's own code path. Four release targets: group chip, grip column, link, bare cell. Each keeps all four rows and sequential <code>data-pos</code> .
<code>python tab_organiser_ext\test_content_js.py</code>	Extension content scripts, 12 checks.
<code>python tab_organiser_ext\test_report_js.py</code>	Extension report scripts, 15 checks.

The headless editor test stubs `/api/live`, so it needs no Chrome profile. The native drag test and the live import both read this machine's real Chrome session files.

9. Troubleshooting

“Chrome session not available: no open Chrome windows found”

Either Chrome has no session yet on this machine, or the session folder is not the Default profile. Check that `%LOCALAPPDATA%\Google\Chrome\User Data\Default\Sessions\` exists, or point the editor at the machine that uses this Chrome profile.

The import list is missing the very newest tabs

Chrome was mid-write on its newest session file and it was skipped. Click the button again after a moment to get the latest complete snapshot.

“Chrome session not available ... (--no-chrome)”

Expected: you started the editor with `--no-chrome`. Drop that flag to use the import.

The report window names and the picker don't match

Normal. Chrome has no window names, so the picker shows `Window 1...N` in tab-strip order while the report keeps whatever names you gave it. The chosen target window is the one whose header you clicked.

The port is already in use

```
python chrome_report_edit_v1.5.py --port 8766.
```

Nothing happened when I clicked the button

The dialog may have failed to open the list — it reports the reason in its own status line. If nothing appears at all, the browser console will show the fetch error; Chrome not running or a `--no-chrome` build explains most cases.

10. Relationship to the report generator

```
chrome_report_v1.4.py    --writes-->  chrome_tabs_Default_v1.4.html
                               |
                        opened in the editor |
                               v
                        chrome_report_edit_v1.5.py
                               ^   |   |
reads live Chrome         |   |   |  +-- Add tabs from window
(SNSS, read-only) -----+   |   |  +---- saves--> any name you type
```

The editor reads and writes the same HTML the generator produces, and additionally reads Chrome's session files to fill gaps in an already-generated report.